

Acceleration of Pattern Matching Algorithms Using Parallel Programming

Paralelização do algoritmo Aho–Corasick em CPUs multi-core de memória compartilhada com
POSIX Threads

Autor	Orientador	Instituição	Finalidade
Matheus Antônio de Castro de Barros	Jeremias Moreira Gomes	IDP — Engenharia de Software	Defesa de Trabalho de Conclusão de Curso (TCC)

Agenda / Roteiro da Apresentação

1. Introdução e Problema

- Contexto de hardware multi-core
- O algoritmo Aho–Corasick
- O regime *memory-bound*
- Objetivos & Questões de Pesquisa (RQs)

2. Fundamentos e Literatura

- Princípios de funcionamento e DFA
- Hierarquia de cache e CPUs híbridas
- Revisão Sistemática da Literatura (RSL)

3. Proposta e Metodologia

- Decisões de design e paralelismo de texto
- Otimização de layout de saída (Flat Layout)
- Ambiente de testes (Intel i5 Alder Lake)

4. Resultados e Conclusão

- Footprint do autômato vs. Throughput
- Escalabilidade e saturação da DRAM
- Resultados negativos (Sharding/Composição 2D)
- Validação e Trabalhos Futuros

Contexto: A Transição para Arquiteturas Multi-core

- **Estagnação do clock em meados de 2000:** Limites térmicos e físicos impediram o aumento infinito da frequência de operação das CPUs.
- **A era multi-core:** A indústria respondeu adicionando múltiplos núcleos de processamento por chip em vez de acelerar um único núcleo.
- **O desafio do software:** Para obter ganhos de desempenho, o software não ganha mais aceleração "de graça"; ele precisa ser explicitamente projetado para execução paralela.
- **Memória Compartilhada:** Sistemas *commodity* modernos compartilham a RAM entre os núcleos, gerando potencial concorrência sobre o barramento.

2005

Início da transição dominante para arquiteturas multi-core

**Frequência →
Núcleos**

O ganho de desempenho deixou de vir do clock e passou a depender do paralelismo explícito

O Algoritmo Aho–Corasick: Eficiente, porém Sequencial

- **Aho-Corasick (AC):** Algoritmo de casamento de múltiplos padrões em tempo linear $O(n + z)$, realizando a busca em uma única passada pelo texto.
- **Aplicações Críticas:** Espinha dorsal de sistemas de segurança como **Snort/Suricata (NIDS)**, YARA (detecção de malware), filtros de spam e bioinformática.
- **A Limitação Sequencial:**
 - O autômato avança estritamente caractere por caractere (da esquerda para a direita).
 - Cada transição depende do estado anterior.
 - Isso torna o algoritmo **intrinsecamente sequencial**, utilizando apenas **1 núcleo** do hardware moderno.

Aplicações do Aho–Corasick

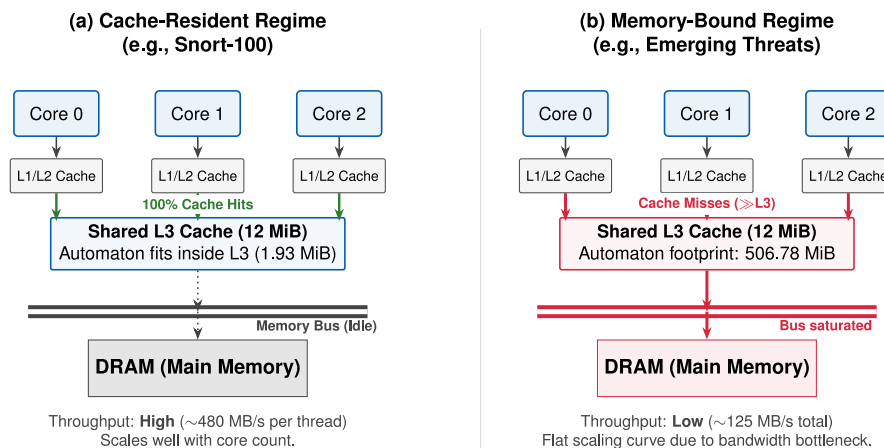
- Snort / Suricata — IDS de rede (NIDS)
- YARA — classificação de malware
- Filtros de spam e antivírus
- Bioinformática — busca de motivos em DNA

1 núcleo

Sequencial por natureza: usa apenas um núcleo, mesmo em CPUs multi-core

Problema e Motivação: O Regime Memory-Bound

- **Crescimento de Assinaturas:** Dicionários de NIDS geram autômatos de **centenas de megabytes**.
- **Estouro da Cache L3 (LLC):** Quando o autômato ultrapassa a cache, entra em *memory-bound*.
- **Saturação do Barramento:** Cada byte processado exige ir à DRAM, gargalando a largura de banda.
- **Falta de Caracterização:** Lacuna na avaliação desse limite em **CPUs híbridas modernas**.



Objetivos do Trabalho

Objetivo Geral: *Projetar, implementar e avaliar uma versão paralela do algoritmo Aho–Corasick para CPUs multi-core de memória compartilhada usando POSIX Threads (pthreads), a fim de acelerar o casamento de múltiplos padrões sobre entradas grandes.*

Objetivos Específicos:

1. Implementar

Construir a versão paralela em C com a API pthreads, explorando o paralelismo intra-entrada sobre um autômato compartilhado.

2. Quantificar

Medir os ganhos em *speedup*, *throughput* e eficiência paralela, sempre contra um baseline sequencial validado.

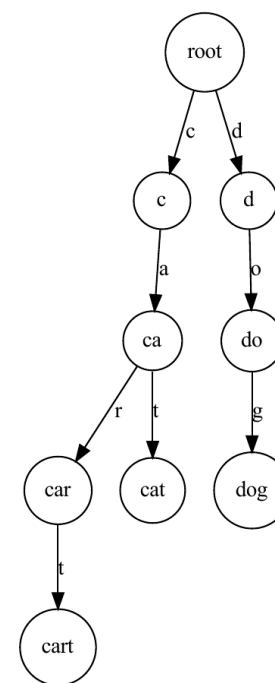
3. Analisar

Avaliar a escalabilidade sob diferentes *workloads*, conjuntos de padrões e números de threads.

Fundamentação: O Aho–Corasick em uma Figura

- **Trie Inicial:** árvore de prefixos gerada a partir do dicionário de padrões (função *goto*).
- **Failure Links:** apontam para o maior sufixo próprio que também é prefixo no dicionário. Evitam o retrocesso no texto (herança do KMP).
- **Compilação para DFA:** tabela de transições plana $\delta(\text{estado}, \text{byte})$.
 - **Exatamente 1 lookup por byte.**
 - Custo: tabela grande ($4 \cdot |Q| \cdot 256$ bytes) → estoura a cache.

Construção do Autômato (Exemplo)



Paralelismo de Memória Compartilhada e Pthreads

Design de Concorrência Eficiente:

- **Compartilhamento de Endereço:** Em CPUs multi-core, as threads compartilham a memória RAM, permitindo que todas acessem um único autômato na memória de forma concorrente.
- **Estado Read-Only:** O autômato de busca é estritamente **somente leitura** após construído.
 - **Hot path livre de locks:** Nenhuma thread worker usa exclusão mútua (*mutex*) ou operações atômicas durante o laço principal de casamento de padrões.
- **POSIX Threads (pthread):** API de baixo nível e padrão da indústria, oferecendo controle fino sobre criação, sincronização de threads e afinidade de CPU.

Arquitetura Proposta

N Threads Workers

↓ (Acesso Concorrente)

1 Autômato Compartilhado (Read-Only)

↓ (Sem Sincronização)

N Listas Privadas de Matches

Métricas: Speedup, Amdahl e Eficiência

Para quantificar a escalabilidade e identificar limites físicos:

- **Speedup (S):** Razão entre o tempo sequencial (T_s) e o tempo paralelo (T_p). Mede a aceleração relativa.

$$S = \frac{T_s}{T_p} \quad (\text{Ideal linear: } S = P \text{ threads})$$

- **Lei de Amdahl:** Determina que a aceleração máxima é limitada pela fração estritamente serial (f) do programa (ex: junção final de resultados, inicialização e escalonamento).

$$S_{max} = \frac{1}{f + \frac{1-f}{P}}$$

- **Eficiência (E):** Mede o aproveitamento real do hardware disponível.

$$E = \frac{S}{P} \times 100\%$$

O Fator Decisivo: Hierarquia de Cache e Núcleos Híbridos

- **Hierarquia de Memória:** Caches L1 e L2 são rápidas e privadas por núcleo. A cache L3 (LLC) é compartilhada e mais lenta. A DRAM (memória física) é ordens de grandeza mais lenta.
- **Saturação por Banda de Memória:** Em regime *memory-bound*, as threads concorrentes passam a competir pela banda limitada do barramento DRAM, achatando a curva de escalabilidade.
- **Arquitetura Híbrida (x86 P/E):**
 - **P-Cores (Performance):** Velozes, com Hyper-Threading (HT).
 - **E-Cores (Efficiency):** Lentos, sem HT, otimizados para consumo.
 - Gera um desequilíbrio intrínseco de carga (estrangulamento em barreira).

Estrutura Híbrida Típica

- **P-Cores:** 2 núcleos (4 threads de hardware)
- **E-Cores:** 8 núcleos (8 threads de hardware)
- **L3 LLC:** 12 MiB compartilhados

"Joelho"

Desvio na escalabilidade ao cruzar P-cores e E-cores

RSL: Questões de Pesquisa e Funil de Seleção

Uma **Revisão Sistemática da Literatura (RSL)** foi conduzida para mapear o estado da arte, guiada por duas questões de pesquisa:

RQ1 — Técnicas e desafios

Quais técnicas de paralelização e desafios associados estão documentados na literatura para implementar o Aho–Corasick com *threads* em CPUs multi-core?

RQ2 — Otimizações em segurança

Quais otimizações específicas do Aho–Corasick são empregadas em cenários com grandes conjuntos de padrões, particularmente no domínio da segurança da informação?

Bases: IEEE Xplore, ACM DL, Scopus, Web of Science · **Período:** 2015–2025 · busca em maio/2025.

Funil de Seleção

196

registros após filtros nas 4 bases



24

estudos únicos relevantes (após duplicatas e triagem de título/resumo)



10

estudos primários — os mais pertinentes às RQs (14 restantes como contexto)

RSL: Estado da Arte e Lacunas

Da análise dos 10 estudos primários selecionados na literatura, emergiram 3 consensos:

1. Divisão por Texto e Overlap

Dividir o texto em chunks com overlap periférico (warm-up) de tamanho $L_{\max} - 1$ é a técnica consolidada para garantir correção sem comunicação (Qu et al., Jepsen et al.).

2. Gargalo de Cache L3

O tamanho do autômato dita a degradação microarquitetural. Propostas como decomposição Head-Body tentam amenizar a pressão de cache de último nível (Lee & Yang).

A Lacuna Científica

Faltava caracterizar a paralelização do Aho–Corasick no **regime memory-bound** (autômato $\gg$ LLC) em CPUs multi-core *commodity*, programadas nativamente em C com pthreads e validadas contra o baseline sequencial.

Proposta: A Decisão Estrutural Única

O design de software proposto no trabalho baseia-se em fases de execução bem delimitadas:

[Fase 1: Construção Sequencial]

Thread mestre constrói a Trie, calcula links de falha e compila a tabela DFA.



[Fase 2: Bloqueio do Autômato (Somente Leitura)]

A tabela de transições torna-se estritamente read-only na memória compartilhada.



[Fase 3: Varredura Concorrente por Workers (Zero Locks)]

Threads leem o texto em paralelo e escrevem matches em buffers locais independentes.



[Fase 4: Consolidação dos Resultados (Merge)]

Mesclagem rápida das listas privadas de ocorrências em uma saída final ordenada.

Particionamento do Texto + Overlap de Borda

- O arquivo de entrada é segmentado em T partes contíguas, uma por thread.
- **O Problema da Fronteira:** Padrões que iniciam em um segmento e terminam no seguinte seriam perdidos.
- **A Solução por Overlap:** Cada worker da thread $i > 0$ inicia a busca $L_{\max} - 1$ bytes antes do início oficial do seu bloco (onde $L_{\max}$ é o comprimento do padrão mais longo).
 - Funciona como um *warm-up* de estado.
 - Garante correspondência **matematicamente idêntica (1:1)** em relação ao algoritmo sequencial, sem comunicação entre threads.

Esquema de Overlap

Segmento Thread 0

$L_{\max} - 1$ (overlap)

Segmento Thread 1

O overlap garante que os padrões que cruzam a fronteira física de blocos sejam lidos e registrados por exatamente um worker.

As Quatro Estratégias Projetadas e Avaliadas

1. Paralelismo de Texto Flat

Chunking uniforme do texto associado ao **Flat Output**

Layout: troca ponteiros espalhados por arrays contíguos de emissão. Minimiza cache misses no matching.

2. Balanceamento Heterogêneo

Duas variantes para evitar lentidão nos E-cores: •

Frequency-Weighted: Partições proporcionais às capacidades dos núcleos. • *Dynamic Dispatch:*

Distribuição por *Bag-of-Tasks*.

3. Sharding de Dicionário

Divisão dos padrões em K sub-dicionários (por prefixo). Reduz o tamanho de cada autômato na esperança de caber na cache L3. (*Hipótese testada*)

4. Composição Bidimensional (2D)

Combinação simultânea de particionamento de texto (chunking) e particionamento de dicionário (sharding). (*Hipótese testada*)

Ambiente Experimental e Workloads

Plataforma de Testes:

- **Processador:** Intel Core i5-1235U (Arquitetura Alder Lake).
 - 2 P-Cores (4 threads de hardware, Hyper-Threading ativo) + 8 E-Cores (8 threads). Total: **12 threads de hardware**.
 - **Cache L3 Compartilhada:** 12 MiB.
 - **Memória:** 31 GiB DDR4 dual-channel (banda compartilhada na casa de dezenas de GB/s — o recurso disputado no regime memory-bound).
- **Compilador & OS:** C11, GCC 13.3.0 (`-O3 -march=native`), Linux Kernel 6.17.

Dicionários de Teste (Escalabilidade de Footprint):

- **Snort-100:** 100 regras → Autômato com **1,93 MiB** Residente na Cache L3
- **Snort-1k:** ~1.000 regras → Autômato com **11,92 MiB** Borda da Cache L3
- **Snort (full):** 4.188 regras → Autômato com **55,23 MiB** Estoura a Cache L3
- **Emerging Threats (ET):** 44.678 regras → Autômato com **506,78 MiB** **42× o tamanho da Cache L3**

Rigor Experimental e Protocolo de Medição

Para afastar desvios comuns em benchmarks acadêmicos de sistemas paralelos:

- **Sweep Automatizado:** Execução programada controlando rigidamente o tempo e o ruído térmico (banco de dados de medições `sweep.db`).
- **Evitando o Efeito Boost:** Processadores móveis sofrem *thermal throttling* sob estresse prolongado. Os dados coletados refletem o **estado sustentado (sob carga)** e estável, não picos frios de rajada.
- **Protocolo por Medição:** 2 execuções de aquecimento (*warm-up*) seguidas de 5 repetições para cada ponto de curva. Reportados throughput médio e coeficiente de variação (CV).
- **Validação Absoluta de Correção:**
 - Verificação do hash **MD5** do fluxo de saída de casamento de padrões para todas as variantes experimentais contra o baseline sequencial (de 1 a 64 threads).
 - Instrumentação via **ThreadSanitizer** para validar a ausência de *data races*.

Estratégias Avaliadas

Todas as variantes compartilham o mesmo autômato e são selecionáveis em tempo de execução. Cada eixo é isolado para atribuir o efeito a uma única causa, sempre comparado ao baseline sequencial.

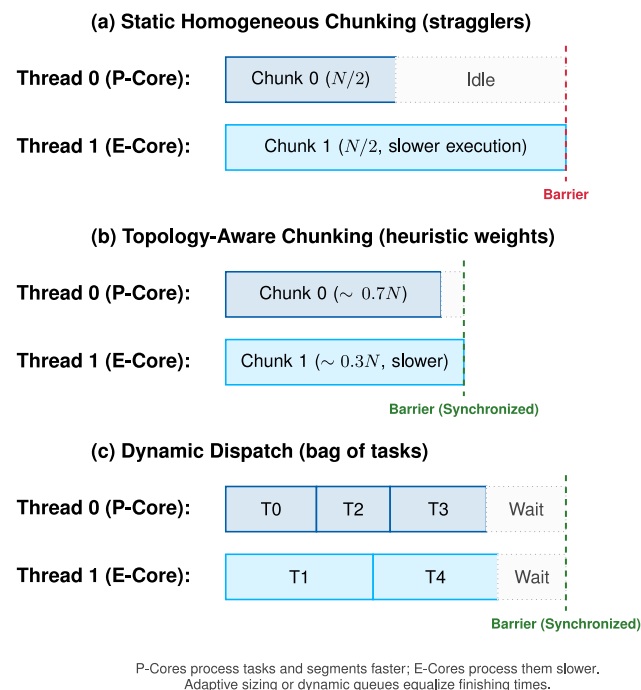
Eixo de otimização	Estratégia
Baseline	Varredura sequencial de thread única (referência de correção e speedup).
Paralelismo de texto	Chunking estático homogêneo com overlap de borda.
	Chunking estático refinado (padding de cache + laço warm-up/owned separado).
	Chunking <i>topology-aware</i> (segmentos ponderados pela frequência do núcleo).
	Dispatch dinâmico por contador atômico (<i>bag of tasks</i>).
Layout de saída	Tabela de saída achatada e contígua (substitui a emissão por ponteiros).
Paralelismo de dicionário	Sharding por prefixo em K sub-autômatos independentes.
Composição	Combinação 2D de sharding de dicionário com chunking de texto.

Estratégias de Balanceamento de Carga

Enfrentando a Assimetria

Processadores híbridos (P-Cores vs E-Cores) geram desequilíbrio intrínseco. Estratégias projetadas:

- **(a) Static Homogeneous:** Divisão igual de tarefas. Gera atrasos massivos de sincronização por conta dos *stragglers* (E-Cores).
- **(b) Topology-Aware:** Divisão heurística (pesos fixos). Melhora muito, mas a perfeição matemática é inatingível na prática.
- **(c) Dynamic Dispatch:** Consumo contínuo de um *Bag-of-Tasks* usando contadores atômicos para máxima adaptabilidade.



Resultados: Impacto do Footprint no Throughput

Throughput sequencial (monothread) medido mantendo o corpus estritamente fixo e alterando apenas o tamanho do autômato de transições:

Dicionário	Padrões	Tamanho	Regime de Cache	Throughput Monothread
Snort-100	100	1,93 MiB	Interno à L3 (Residente)	483,1 MB/s
Snort-1k	~1.000	11,92 MiB	Limite da L3	337,3 MB/s
Snort (full)	4.188	55,23 MiB	Extrapola a L3 (Estouro)	240,7 MB/s
Emerging Threats	44.678	506,78 MiB	Estouro crítico (42× L3)	125,6 MB/s

-74,0%

Degradação no throughput monothread apenas variando o tamanho do dicionário

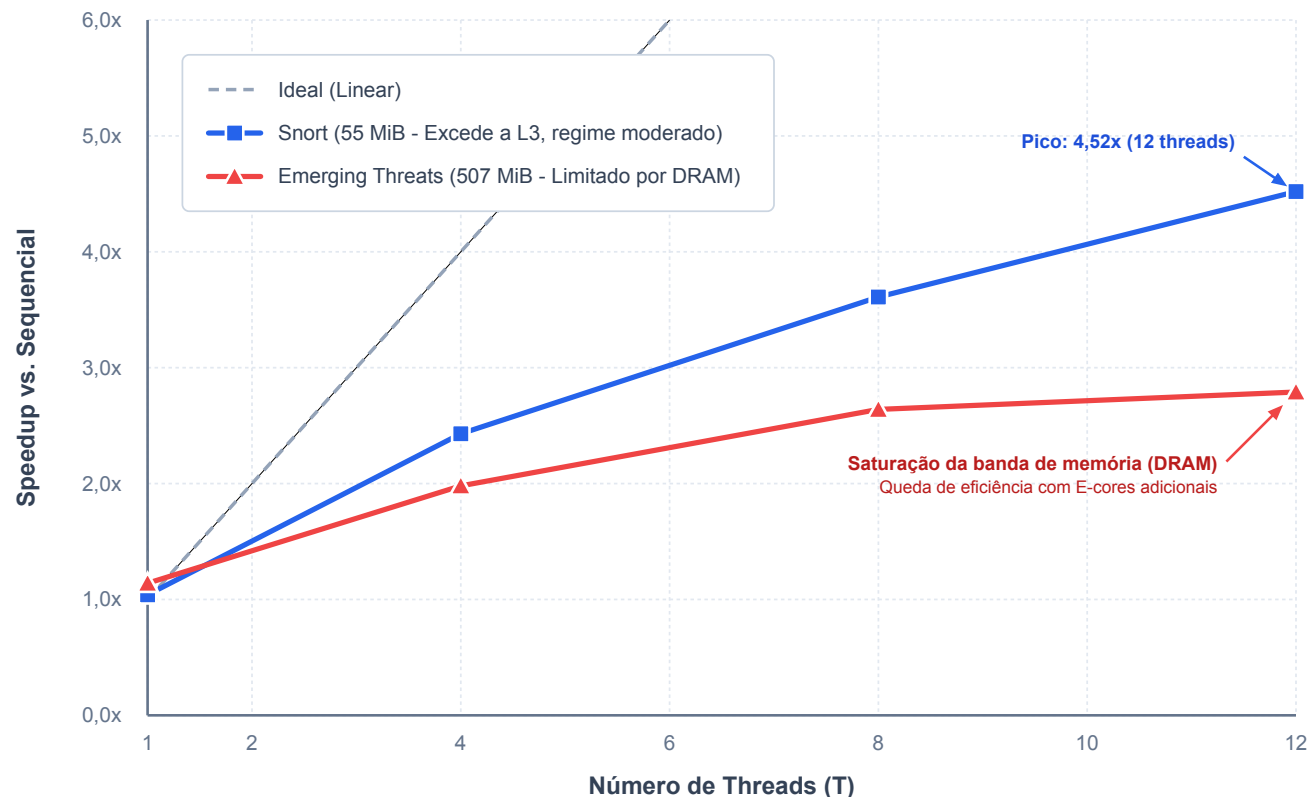
A queda acentuada comprova a natureza do regime: uma vez fora da cache, cada transição vira um acesso à DRAM.

É o footprint, não o texto

Mesmo dicionário, corpora diferentes: **241,0** (Enron) × **248,4 MB/s** (SimpleWiki). O conteúdo do texto quase não importa — o tamanho do autômato domina.

Escalabilidade do Paralelismo de Texto

Escalabilidade do Paralelismo de Texto por Dicionário



Regime Moderado: Snort (55 MiB)

- Excede a L3, mas a banda de DRAM ainda não é o limite.
- Escala de forma estável até 12 threads.
- **Pico: 4,52x** (e 4,79x no corpus canônico).
- Eficiência paralela de **37,7%**.

Regime Limitado: ET (507 MiB)

- Ocorre saturação precoce do barramento.
- **Pico: 2,79x** obtido com 12 threads.
- Em Enron (canal principal) o pico ocorre com 6 threads (**2,85x**) e **regride** ao injetar mais threads.
- Eficiência paralela restrita a **23,3%**.

Resultados: Otimização Flat Output Layout

Substituição da lista encadeada de ponteiros na emissão de ocorrências por um array contíguo indexável. Benefício testado de forma isolada em thread única:

Regime (Dicionário)	Tamanho	Baseline (MB/s)	Flat Layout (MB/s)	Ganho Líquido
Snort-100 (<< L3)	1,93 MiB	483,1	488,1	+1,0%
Snort (full) (> L3)	55,23 MiB	240,7	252,0	+4,7%
Emerging Threats (>> L3)	506,78 MiB	125,6	142,0	+13,0%

+13,0%

Ganho obtido sem introduzir overhead de sincronização paralela

Conclusão: O ganho da otimização de layout de dados é proporcional à dificuldade do problema. É mais de 10x superior no pior cenário (regime memory-bound) ao remover cache misses do percurso de ponteiros.

Resultados: Balanceamento em CPUs Híbridas

Diferença de tempo de execução entre núcleos rápidos (Performance) e lentos (Efficiency) causa flutuações e barreiras de atraso (*stragglers*). Medição da variação temporal a 12 threads:

Estratégia de Partição	Throughput Médio	Coeficiente Variação (CV)
Static Homogeneous (Padrão)	1.064,7 MB/s	39,9%
Static Chunking (Com alinhamento)	902,0 MB/s	52,9%
Topology-aware (Frequency-weighted)	879,8 MB/s	1,7%
Dynamic Dispatch (Bag of Tasks)	864,4 MB/s	3,3%
Flat Chunking (Particionado uniforme)	1.007,2 MB/s	0,4%

Foco na Previsibilidade

O particionamento balanceado reduz a variação de até **52,9% para menos de 2%**. **Impacto real:** Não altera a velocidade intrínseca da transição de autômato, mas remove tempo morto de sincronização em barreira. Garante latência determinística no pior caso.

Resultados Negativos: Sharding de Dicionário

Hipótese Inicial:

Dividir o grande autômato em sub-autômatos menores (shards por prefixo) aumentaria a localidade de cache L3 e superaria o custo da computação extra.

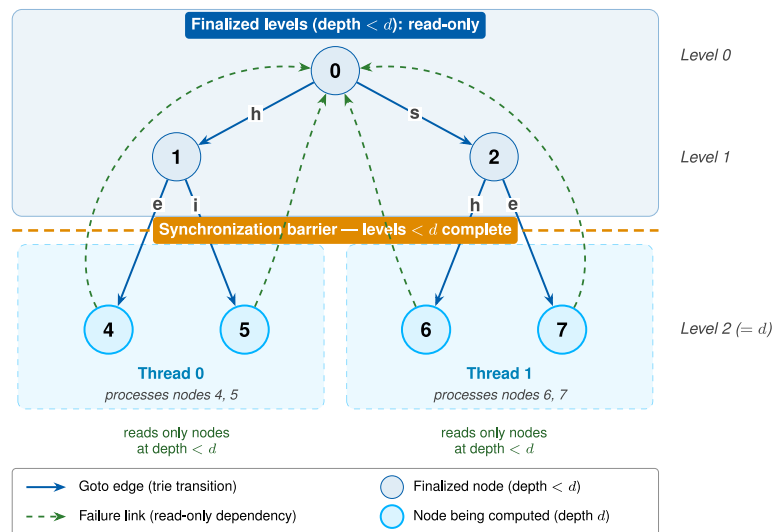
Estratégia de Busca	Throughput	vs. Sequencial	vs. Paralelismo Texto
Dictionary Sharding (Prefixo)	327,2 MB/s	1,37x	0,34x
Paralelismo de Texto Flat	976,1 MB/s	4,07x	1,00x

X Falsificada

Hipótese de Sharding e Composição 2D

Por que falhou? A divisão do dicionário força o processamento a varrer o texto múltiplas vezes (K shards). A banda de memória satura pelo aumento de tráfego de leitura de dados, neutralizando qualquer ganho local de cache L3.

Construção Paralela: Level-Synchronous BFS



O Desafio da Dependência

Calcular o *failure link* $f(v)$ de um estado exige saber o $f(u)$ do seu ancestral. Distribuir estados aleatoriamente entre threads geraria *race conditions* ou exigiria *locks* pesados.

A Solução (Sem Locks)

- **Processamento por Níveis:** A Trie é fatiada por profundidade.
- **Read-only Ascendente:** Dependências (setas verdes) apontam apenas para níveis já calculados e estáticos.
- **Sincronização:** Barreiras garantem consistência antes de descer de nível.

Avaliação End-to-End e Construção Paralela

Análise global integrando todas as fases do pipeline (construção do DFA + casamento de padrões + mesclagem de saídas) executado sob o corpus massivo de ~10,6 GiB:

Dicionário	Fase Sequencial	Melhor Fase Paralela	Speedup E2E Real
Snort (full) (55 MiB)	45.174 ms	10.027 ms	4,50x
Emerging Threats (507 MiB)	86.998 ms	31.517 ms	2,76x

4,50x

Aceleração global de ponta a ponta obtida no regime moderado

2,76x

Aceleração global de ponta a ponta obtida no regime crítico limitado

Construção paralela do autômato

Só compensa no dicionário grande (Emerging Threats): **1,55x** em 8 threads. Em scans de múltiplos GB, a construção é menos de 1% do tempo total — por isso o melhor resultado memory-bound usa *build* sequencial.

Conclusão

*Achado central: o principal obstáculo para escalar o Aho–Corasick em escala de IDS é **microarquitetural** — estouro de cache e banda de DRAM —, não a lógica de casamento. A resposta correta é paralelizar o **texto** com um layout de emissão amigo da cache.*

Objetivos atendidos

- **Implementação** em C/pthreads, com correção validada em todos os workloads.
- **Ganhos quantificados:** 4,79× / 4,52× de busca; eficiências de 37,7% e 23,3%.
- **Escalabilidade analisada:** 7 contagens de thread, 3 corpora, autômatos cobrindo faixa de 42× a L3.

Contribuições

- **Caracterização do regime memory-bound** em CPU multi-core commodity, com identificação do ponto de saturação da DRAM.
- **Flat output layout:** ganho limpo de até +13,0% onde o problema é mais difícil.
- **Resultados negativos:** sharding e composição 2D falsificados, fechando uma região do espaço de design.

Ameaças à Validade e Trabalhos Futuros

Ameaças à Validade

- **Throttling Térmico:** Hardware móvel restringe pico contínuo; mitigado com execução sustentada e repetições controladas.
- **Arquitetura Única:** Avaliado em um processador híbrido. Todavia, a tese do gargalo de DRAM estende-se a servidores cujo autômato exceda a LLC compartilhada.
- **Telemetria Indireta:** Diagnósticos baseados em throughput temporizado; ausência de amostragem por PMUs na placa.

Trabalhos Futuros

- **Contadores de Hardware (PMU):** Registrar cache-misses e stalls de pipeline por telemetria direta para consolidar o modelo microarquitetural.
- **Avaliação de Servidores:** Executar em processadores de classe enterprise com maior banda de memória e caches L3 estendidas.
- **Sharding por Densidade:** Agrupar shards por densidade real de transições frequentes e não meramente por primeiro byte.

Obrigado

Matheus Antônio de Castro de Barros

Orientador: Jeremias Moreira Gomes

Trabalho de Conclusão de Curso — IDP Engenharia de Software